

- Description

This module manages a smart irrigation system, ensuring that:

1. **The water requirement for a field is calculated based on soil moisture, temperature, and crop type.**
2. **If the moisture level is above the threshold, irrigation is skipped to conserve water.**
3. **If the moisture is too low, the system calculates and dispenses the required water volume.**
4. **A log of all irrigation events, including water usage and conservation, is maintained.**

This system optimizes water usage by preventing over-irrigation while ensuring adequate crop hydration.

- Specification

module SmartIrrigationSystem

var currentMoisture, moistureThreshold, temperature, maxWaterPerSession, waterToApply: real

var cropType: string

var irrigationLog: sequence of string

process irrigateField(cropType: string, currentMoisture: real, temperature: real) waterToApply: real

functional scenario form:

cropType notin(1,2,3) && Error: Unknown crop type.

||

currentMoisture >= moistureThreshold && waterToApply = 0, irrigationLog = irrigationLog $\oplus$ ("Irrigation Skipped: " + cropType), No irrigation needed.

||

currentMoisture < moistureThreshold && waterToApply = min(cropWaterNeeds[cropType] * (1 + (temperature - 25) * 0.02), maxWaterPerSession), irrigationLog = irrigationLog $\oplus$ ("Irrigation Applied: " + cropType), Irrigation applied with calculated water amount.

end process

end module

- Java Code

```
package code;
import java.util.*;
class IrrigationSystem {
    private double moistureThreshold;
    private double maxWaterPerSession;
    private Map<String, Double> cropWaterNeeds;
    private List<String> irrigationLog;
```

```

    public IrrigationSystem(double moistureThreshold, double maxWaterPe
rSession) {
        this.moistureThreshold = moistureThreshold;
        this.maxWaterPerSession = maxWaterPerSession;
        this.irrigationLog = new ArrayList<>();
        this.cropWaterNeeds = new HashMap<>();
        cropWaterNeeds.put("Wheat", 10.0);
        cropWaterNeeds.put("Corn", 15.0);
        cropWaterNeeds.put("Rice", 20.0);
    }
    public void irrigateField(String cropType, double currentMoisture,
double temperature) {
        if (!cropWaterNeeds.containsKey(cropType)) {
            System.out.printf("Error: Unknown crop type '%s'.%n", cropT
ype);
            return;
        }
        if (currentMoisture >= moistureThreshold) {
            System.out.printf("Moisture level sufficient (%.1f%%). No i
rrigation needed.%n", currentMoisture);
            irrigationLog.add(String.format("Irrigation Skipped: Crop =
%s, Moisture = %.1f%%", cropType, currentMoisture));
            return;
        }
        double baseWaterNeed = cropWaterNeeds.get(cropType);
        double adjustedWater = baseWaterNeed * (1 + (temperature - 25)
* 0.02);
        double waterToApply = Math.min(adjustedWater, maxWaterPerSessio
n);
        System.out.printf("Irrigating %s field: Applying %.1f liters of
water.%n", cropType, waterToApply);
        irrigationLog.add(String.format("Irrigation Applied: Crop = %s,
Water = %.1f L, Moisture = %.1f%%",
                                cropType, waterToApply, current
Moisture));
    }
    public void printIrrigationLog() {
        System.out.println("\nIrrigation Log:");
        for (String log : irrigationLog) {
            System.out.println(log);
        }
    }
}
public class c23_SmartIrrigationSystem {

```

```

    public static void main(String[] args) {
        IrrigationSystem system = new IrrigationSystem(40.0, 25.0);
        // testcase-VT:
        //     system.irrigateField("Wheat", 35.0, 30.0);
        //     system.irrigateField("Corn", 45.0, 28.0);
        //     system.irrigateField("Rice", 25.0, 35.0);
        //     system.irrigateField("Soybean", 30.0, 32.0);

        // testcase-FT:
        //     system.irrigateField("??", -1.7976931348623157E308, -
        1.7976931348623157E308);
        //     system.irrigateField("", -1.12091454291415E308, -
        1.12091454291415E308);
        //     system.irrigateField("'&' ", 5.005341669616643E307, 5.005341669
        616643E307);
        //     system.irrigateField("GGGGW", -1.374706514894712E308, -
        1.7976931348623157E308);
        // testcase-Z3:
        //     system.irrigateField("Wheat", 35, 30);
        //     system.irrigateField("Corn", 45, 28);
        //     system.irrigateField("Rice", 25, 35);
        //     system.irrigateField("Soybean", 30, 32);

        // testcase-UVT:
        system.irrigateField("Wheat", 35.0, 30.0);
        system.irrigateField("Corn", 45.0, 28.0);
        system.irrigateField("Rice", 25.0, 35.0);
        system.irrigateField("Soybean", 30.0, 32.0);
        system.printIrrigationLog();
    }
}

```

- Coverage report

1. Test case number: 4
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c23_SmartIrrigationSystem.java	100.0 %	6	0	6
> c23_SmartIrrigationSystem		0	0	0
> IrrigationSystem	100.0 %	6	0	6

3. Vibration testing instruction coverage: 98.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.2 %	198	16,571	16,769
▼ src	1.2 %	198	16,571	16,769
▼ code	1.2 %	198	16,571	16,769
▼ c23_SmartIrrigationSystem.java	98.5 %	198	3	201
> c23_SmartIrrigationSystem	90.6 %	29	3	32
> IrrigationSystem	100.0 %	169	0	169

4. Fuzz testing branch coverage: 33.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.2 %	2	840	842
▼ src	0.2 %	2	840	842
▼ code	0.2 %	2	840	842
▼ c23_SmartIrrigationSystem.java	33.3 %	2	4	6
> IrrigationSystem	33.3 %	2	4	6
> c23_SmartIrrigationSystem		0	0	0

5. Fuzz testing instruction coverage: 48.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.6 %	97	16,672	16,769
▼ src	0.6 %	97	16,672	16,769
▼ code	0.6 %	97	16,672	16,769
▼ c23_SmartIrrigationSystem.java	48.3 %	97	104	201
> IrrigationSystem	40.2 %	68	101	169
> c23_SmartIrrigationSystem	90.6 %	29	3	32

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c23_SmartIrrigationSystem.java	100.0 %	6	0	6
> c23_SmartIrrigationSystem		0	0	0
> IrrigationSystem	100.0 %	6	0	6

7. Z3 instruction coverage: 98.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.2 %	198	16,571	16,769
▼ src	1.2 %	198	16,571	16,769
▼ code	1.2 %	198	16,571	16,769
▼ c23_SmartIrrigationSystem.java	98.5 %	198	3	201
> c23_SmartIrrigationSystem	90.6 %	29	3	32
> IrrigationSystem	100.0 %	169	0	169

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	836	842
▼ src	0.7 %	6	836	842
▼ code	0.7 %	6	836	842
▼ c23_SmartIrrigationSystem.java	100.0 %	6	0	6
> c23_SmartIrrigationSystem		0	0	0
> IrrigationSystem	100.0 %	6	0	6

9. Unimproved V-method instruction coverage: 98.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.2 %	198	16,571	16,769
▼ src	1.2 %	198	16,571	16,769
▼ code	1.2 %	198	16,571	16,769
▼ c23_SmartIrrigationSystem.java	98.5 %	198	3	201
> c23_SmartIrrigationSystem	90.6 %	29	3	32
> IrrigationSystem	100.0 %	169	0	169